Containers

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Universidade de Aveiro

Introduction

What Are Containers?

A **container** is a standard, executable unit of software that packages an application's code together with all its runtime dependencies — libraries, configuration files, and system tools.

This package is **isolated**, ensuring the application runs uniformly and consistently on any compatible host.

Analogy: A container is like a standardized shipping container. It does not matter what is inside; it can be handled by any compatible ship, truck, or crane (host machine).

Terminology

Before we dive in, let us define some key terms.

- **Image:** A read-only, inert template containing an application and its dependencies. Think of it as a **blueprint** or a class in object-oriented programming.
- **Container / Instance:** A runnable **instance** of an image. This is the actual, living application (like an object created from a class).
- **Registry:** A storage system for container images. **Docker Hub** is a popular public registry.
- **Engine / Runtime:** The software that builds, runs, and manages containers (e.g., Docker Engine, Podman).
- **Volume:** A mechanism for persisting data outside a container's ephemeral filesystem.

The Problem: “It Works on My Machine!”

Every developer has faced this classic problem:

- Your application works perfectly on your laptop (which has Python 3.9, a specific library version, and runs Debian).
- When you give it to a colleague (who has Python 3.8 and runs macOS) or deploy it to a server (running an older OS), it fails.

These differences in environments create a massive challenge for software portability and reproducibility.

The Solution: Containers

Containers solve this problem by bundling **everything the application needs** into a single, self-contained package.

- The application code.
- The language runtime (e.g., Python 3.9, Node.js 20).
- All required libraries and their exact versions.
- System-level dependencies and configuration.

The container runs identically on a developer's laptop, a CI/CD server, or a production cloud instance.

Container Fundamentals

How Isolation is Achieved: Namespaces

Containers run at **full hardware speed** because they are just isolated processes on the host's kernel. The isolation is provided by **Linux Namespaces**.

Namespaces virtualize system resources for a process, making it appear to have its own private copy. Key namespaces include:

- **PID:** Isolates process IDs. Inside the container, your app is PID 1.
- **NET:** Provides an isolated network stack (IP addresses, routing tables).
- **MNT:** Isolates filesystem mount points.
- **UTS:** Isolates hostname and domain name.
- **USER:** Maps container UIDs/GIDs to different host UIDs/GIDs.

How Resources are Managed: Cgroups

To prevent one container from consuming all system resources, the Linux kernel uses **Control Groups (cgroups)**.

Cgroups allow the host to limit and monitor the resources a container can use:

- CPU usage (e.g., limit to 1 CPU core).
- Memory (e.g., limit to 512 MB of RAM).
- Disk I/O bandwidth.
- Network bandwidth.

Analogy: Namespaces are the **walls** between apartments. Cgroups are the **utility meters and circuit breakers**, ensuring no single tenant can use all of the building's resources.

VMs vs. Containers: Architecture

- **Virtual Machines (VMs)** virtualize the **hardware**. Each VM includes a full copy of a guest OS and kernel. They are heavyweight and take minutes to boot.
- **Containers** virtualize the **operating system**. They share the host system's kernel and are lightweight, booting in seconds.

VMs vs. Containers: Comparison

Feature	Virtual Machines	Containers
Analogy	**Houses:** Fully self-contained.	**Apartments:** Share building infrastructure.
Abstraction	Hardware Virtualization	OS Virtualization
Size	Gigabytes (GB)	Megabytes (MB)
Startup Time	Minutes	Seconds or less
Performance	Low to Medium overhead	Very Low (Near-native)
Resource Usage	Higher (Full OS per VM)	Lower (Shared OS Kernel)
Isolation	**Strong** (Hardware level)	**Good** (Process level)
Portability	Portable (but	**Extremely**

The Container Image and Its Layers

An **image** is a read-only template built from a series of stacked **layers**. Each instruction in a `Dockerfile` creates a new layer.

- **Base layer:** The starting OS image (e.g., `alpine`, `ubuntu`).
- **Intermediate layers:** Each `RUN`, `COPY`, or `ADD` instruction adds a layer.
- **Top layer (container layer):** A thin, writable layer added when a container starts.

This makes builds fast and disk usage efficient, as multiple images can share common base layers.

Persistent Data: Volumes

By default, a container's filesystem is **ephemeral** — it is deleted when the container is removed.

To save data permanently, you use **volumes**:

- **Named volumes:** Managed by Docker/Podman. Best for databases and persistent application data. Example: `docker volume create mydata`.
- **Bind mounts:** Map a specific host directory into the container. Best for development, where you want live code changes. Example: `-v ./src:/app/src`.

Key rule: Never store important data only inside a container's writable layer.

Container Networking and DNS

The container engine creates a **virtual bridge network**. Containers on the same network get a private IP and can communicate.

- **Port Mapping:** To expose a container's service to the outside world, you map a host port to a container port (e.g., `-p 8080:80`).
- **Internal DNS:** When using Docker Compose, each service can reach another using its service name as a hostname. Your webapp code can simply connect to `http://database:5432` to reach the database container.

The OCI Standard

The **Open Container Initiative (OCI)** defines industry standards for container formats and runtimes.

- **Image Specification:** Defines the format of container images, ensuring any OCI-compliant image works with any OCI-compliant runtime.
- **Runtime Specification:** Defines how to run a container (the reference implementation is `runc`).
- **Distribution Specification:** Defines how to push/pull images to/from registries.

Because Docker, Podman, and other tools all follow OCI standards, images built with one tool work seamlessly with another.

Docker

Introducing Docker

Docker is the platform that popularized containers. It provides a simple set of tools to build, ship, and run any application, anywhere.

- **Docker Engine:** The background service (daemon) that manages containers.
- **Docker CLI:** The command-line tool you use to interact with the Docker Engine.
- **Docker Hub:** A public registry of pre-built container images.
- **Docker Desktop:** A GUI application for Windows and macOS that bundles the Engine, CLI, and Compose.

Docker Architecture

Docker uses a **client-server** model:

1. The **Docker CLI** (client) sends commands to the **Docker Daemon** (`dockerd`).
2. The daemon does the heavy lifting: building images, running containers, managing networks and volumes.
3. The daemon pulls images from a **Registry** (e.g., Docker Hub) when needed.

The daemon runs as `root` and listens on a Unix socket. This is a key architectural difference from Podman.

Common Docker Commands

Command	Description
`docker run [image]`	Creates and starts a new container from an image.
`docker ps`	Lists running containers. `ps -a` lists all.
`docker stop [id/name]`	Stops a running container gracefully.
`docker rm [id/name]`	Removes a stopped container.
`docker logs [id/name]`	Fetches the logs (stdout/stderr) from a container.
`docker exec -it [id] sh`	Opens an interactive shell inside a running container.
`docker pull [image]`	Downloads an image from a

The Dockerfile: Core Instructions

A `Dockerfile` is a recipe for building a container image. Here are the most common instructions:

- `FROM`: Specifies the base image to build upon (e.g., `ubuntu:22.04`).
- `WORKDIR`: Sets the working directory for subsequent commands.
- `COPY`: Copies files or directories from the host into the image.
- `RUN`: Executes a command during the build (e.g., `RUN apt-get install -y nginx`).

- `CMD`: Provides the default command to run when a container starts.
- `ENTRYPOINT`: Configures the container to run as an executable.
- `EXPOSE`: Documents which ports the container listens on at runtime.
- `ENV`: Sets persistent environment variables.
- `ARG`: Defines build-time variables (not available at runtime).

Dockerfile Example: A Logging Service

This simple `Dockerfile` creates a service whose only job is to print a timestamp every 5 seconds. This is useful for testing the `docker logs` command.

```
# Use a minimal base image
FROM alpine:latest

# The command to execute when the container starts.
# An infinite loop that prints the date and sleeps.
CMD ["sh", "-c", \
     "while true; do \
       echo \"[LOG] Server running at $(date)\"; \
       sleep 5; \
     done"]
```

To build and run it:

```
# Build the image and tag it
$ docker build -t logging-service .

# Run the container in detached mode
$ docker run -d --name logger logging-service

# Follow the logs in real time
$ docker logs -f logger
```

Dockerfile Best Practices

Writing efficient Dockerfiles makes images **smaller, faster to build, and more secure**.

- **Use small base images:** Prefer `alpine` or `-slim` variants over full ubuntu/debian images.
- **Combine `RUN` commands:** Each `RUN` creates a layer. Chain commands with && to reduce layers.
- **Order instructions by change frequency:** Put rarely-changing instructions (e.g., `apt install`) before frequently-changing ones (e.g., `COPY . .`) to maximize layer cache hits.
- **Use `.dockerignore`:** Exclude files like `.git/`, `node_modules/`, and build artifacts from the build context.

Multi-Stage Builds

Multi-stage builds let you use multiple FROM statements in one Dockerfile to **separate build tools from the final runtime image**.

```
# Stage 1: Build the application
FROM golang:1.22 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp .

# Stage 2: Create a minimal runtime image
FROM alpine:latest
COPY --from=builder /app/myapp /usr/local/bin/
CMD ["myapp"]
```

The final image contains **only the compiled binary**, not the

Docker Compose

Docker Compose: Overview

Docker Compose is a tool for defining and managing **multi-container applications** using a single YAML file.

Instead of running multiple `docker run` commands with complex flags, you describe your entire application stack in a `compose.yml` file and manage it with simple commands:

```
$ docker compose up -d      # Start all services
$ docker compose down        # Stop and remove all
$ docker compose logs -f     # Follow logs from all
$ docker compose ps          # List running service
```

Compose File: Key Directives

A `compose.yml` file uses these common keys:

- `services`: The root key where all your application services are defined.
- `image`: Specifies a pre-built image from a registry.
- `build`: Specifies a path to a `Dockerfile` to build the service's image.

- `ports`: Maps ports from the host to the container (e.g., `"8080:80"`).
- `volumes`: Mounts host paths or named volumes into the container.
- `environment`: Sets environment variables for the service.
- `depends_on`: Defines dependencies between services, controlling startup order.
- `networks`: Assigns the service to one or more custom networks.
- `restart`: Sets the restart policy (e.g., `unless-stopped`, `always`).

Compose Example 1: Building a Custom NGINX Image

This example shows how to package your website's files directly into a custom image.

Required File Structure:

```
.
├── compose.yml
├── Dockerfile
└── my-website/
    └── index.html
```

Dockerfile

```
# Use the official NGINX image as a base
FROM nginx:alpine

# Copy our custom webpage into the image's web root
COPY ./my-website /usr/share/nginx/html
```

compose.yml

```
services:
  webserver:
    build: .
    ports:
      - "8080:80"
```

Example 1: Explanation

In this method, we create a **self-contained, portable image** that includes our application code.

1. When you run `docker compose up`, the `build: .` directive tells Compose to look for a `Dockerfile` in the current directory.
2. The `Dockerfile` starts from the standard `nginx:alpine` base image.
3. The `COPY` instruction copies the local `./my-website` folder into the image at `/usr/share/nginx/html`.
4. A new, custom image is created containing both NGINX and your webpage.
5. A container is started from this new image with port 8080 mapped to port 80.

Key Concept: The application and its code are bundled together. This is ideal for **production deployments**, as the resulting image is a consistent, immutable artifact that can be run anywhere.

Compose Example 2: Using a Volume to Serve Content

This example uses a standard NGINX image and injects content using a bind mount volume.

Required File Structure:

```
.
├── compose.yml
└── my-website/
    └── index.html
```

compose.yml

```yaml
services:
  webserver:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./my-website:/usr/share/nginx/html
```

(No Dockerfile is needed for this method)

Example 2: Explanation

This method keeps your code on the host machine and dynamically links it into the container.

1. The `image: nginx:alpine` directive pulls the standard NGINX image from Docker Hub. No custom image is built.
2. A container is started from this standard image.
3. The `volumes` directive creates a live link between the `./my-website` folder on your host and `/usr/share/nginx/html` inside the container.
4. NGINX reads files directly from your host machine's disk.

Key Concept: The container is stateless, and the code lives on the host. If you change your `index.html` file, the change is reflected **instantly** without rebuilding or restarting. This is ideal for **local development**.

Compose Example 3: NGINX with a Varnish Cache

This advanced example orchestrates two services: an NGINX web server and a Varnish cache that sits in front of it to speed up content delivery.

Required File Structure:

```
.
├── compose.yml
└── varnish/
    └── default.vcl
```

`varnish/default.vcl` (Varnish Configuration)

```
vcl 4.1;

// Define the backend server Varnish will fetch con
// 'nginx' is the service name from compose.yml.
backend default {
    .host = "nginx";
    .port = "80";
}
```

compose.yml

```yaml
services:
  # The Varnish cache, exposed to the outside world
  cache:
    image: varnish:stable
    volumes:
      - ./varnish:/etc/varnish
    ports:
      - "8080:80"
    depends_on:
      - nginx

  # The NGINX web server, internal only
  nginx:
    image: nginx:alpine
    # No ports: only accessible from within the Doc
```

Example 3: Explanation

This setup demonstrates a realistic, multi-tier architecture where services communicate internally.

1. The `compose.yml` defines two services: `cache` (Varnish) and `nginx`.
2. Only the `cache` service exposes a port (`8080`) to the host. The `nginx` service is completely internal.
3. The Varnish configuration references the hostname `nginx`, which Docker's **internal DNS** resolves to the private IP of the `nginx` container.

The Request Flow:

`Browser -> Host:8080 -> Varnish (Cache) -> NGINX (Origin)`

On the first request, Varnish fetches the page from `nginx` and caches it. Subsequent requests are served directly from cache, which is extremely fast.

Key Concept: This demonstrates **service discovery** and a **reverse proxy** pattern, a fundamental building block in web architecture.

Container Ecosystem

The Origin: Linux Containers (LXC)

Before Docker, there was **LXC** (2008).

- LXC is a user-space interface for the Linux kernel's containment features (namespaces and cgroups).
- It provides a lower-level set of tools for creating and managing containers.
- LXC containers typically run a full `init` system and are used for isolating entire operating systems — they behave more like lightweight VMs than application containers.

LXC proved that OS-level virtualization was practical. Docker took this foundation and made it accessible to application developers.

Docker: The De Facto Standard

Docker (2013) took the underlying technology of LXC and built a high-level, developer-friendly ecosystem around it.

- Introduced portable, layered images via the `Dockerfile`.
- Created a centralized registry (Docker Hub) for sharing images.
- Focused on **application-centric** containers: one process per container.
- This philosophy became a cornerstone of the **microservices architecture**.

Docker's main limitation is its **daemon-based architecture**: the `dockerd` daemon runs as root, which can be a security concern in multi-tenant or hardened environments.

Podman: The Modern Alternative

Podman (Pod Manager) is a container engine developed by **Red Hat** as a direct, drop-in replacement for Docker.

- First released in 2018, now widely adopted in enterprise Linux distributions (RHEL, Fedora, CentOS Stream).
- Fully **OCI-compliant**: uses the same image format and runtime standards as Docker.
- Included by default in RHEL 8+ and Fedora, where Docker is no longer shipped.

Podman Architecture: Daemonless

The most important architectural difference between Podman and Docker is that Podman has **no central daemon**.

- **Docker:** CLI –> `dockerd` (daemon, runs as root) –> `containerd` –> `runc`
- **Podman:** CLI –> `conmon` (per-container monitor) –> `runc`

Each container is a **direct child process** of the Podman command (fork/exec model). If Podman exits, the containers keep running under `conmon`.

This eliminates the single point of failure that the Docker daemon represents.

Podman: Rootless Containers

Podman was designed from the ground up to run containers **without root privileges**.

- Containers run in the user's own namespace, with no escalated permissions.
- Uses **user namespaces** to map container UID 0 (root) to an unprivileged UID on the host.
- A compromised container cannot gain root access to the host.

```
# No sudo required
$ podman run --rm -it alpine sh

# Check: inside the container you are "root",
# but on the host you are your regular user
$ podman top -l user huser
```

Podman: CLI Compatibility

Podman's command-line interface is **intentionally identical** to Docker's. You can use the same commands you already know:

```
$ podman pull nginx:alpine
$ podman run -d --name web -p 8080:80 nginx:alpine
$ podman ps
$ podman logs web
$ podman stop web
$ podman rm web
$ podman build -t myapp .
$ podman images
```

On many systems, you can simply create an alias:

```
$ alias docker=podman
```

Podman: Pods

Podman introduces the concept of **pods**, inspired by Kubernetes.

A pod is a group of containers that:

- Share the same **network namespace** (same IP address, same `localhost`).
- Share the same **IPC namespace**.
- Can communicate over `localhost` without port mapping.

```
# Create a pod with port mapping
$ podman pod create --name webapp -p 8080:80

# Add containers to the pod
$ podman run -d --pod webapp --name web nginx:alpi
$ podman run -d --pod webapp --name api my-api-imag
```

Podman Compose

For multi-container applications, Podman supports Compose files through two approaches:

1. **`podman compose` (built-in, Podman 4.7+):**

```
$ podman compose up -d
$ podman compose down
```

2. **`podman-compose` (third-party Python tool):**

```
$ pip install podman-compose
$ podman-compose up -d
```

Both read standard `compose.yml` / `docker-compose.yml` files. Most Compose files work without modification.

Podman: Systemd Integration

Podman can generate **systemd unit files** to manage containers as system services. This means containers start at boot and are monitored by the init system.

```
# Generate a systemd unit for a running container
$ podman generate systemd --new --name web > web.se

# Install and enable the service (rootless)
$ mkdir -p ~/.config/systemd/user/
$ mv web.service ~/.config/systemd/user/
$ systemctl --user daemon-reload
$ systemctl --user enable --now web.service
```

This replaces Docker's `restart: always` policy with a proper, OS-native service manager.

Podman: Kubernetes YAML Generation

Podman can also generate and consume **Kubernetes YAML** directly, bridging local development and production deployment:

```
# Generate Kubernetes YAML from a running pod
$ podman generate kube webapp > webapp.yml

# Deploy from Kubernetes YAML locally
$ podman play kube webapp.yml

# Tear it down
$ podman play kube --down webapp.yml
```

This makes the transition from local Podman development to a Kubernetes cluster much smoother.

Docker vs. Podman: Comparison

Feature	Docker	Podman
Architecture	Client-server (daemon)	Daemonless (fork/exec)
Root required	Yes (daemon runs as root)	No (rootless by default)
CLI	`docker ...`	`podman ...` (compatible)
Compose	`docker compose`	`podman compose`
Pods	Not supported	Supported (Kubernetes-style)
Systemd	Not integrated	Native integration
Kubernetes YAML	Not supported	Generate and play
Desktop GUI	Docker Desktop	Podman Desktop
OCI Compliant	Yes	Yes
Default in RHEL	No	Yes

When to Use Docker vs. Podman

Choose Docker when:

- You are learning containers for the first time (larger community, more tutorials).
- Your team or CI/CD pipeline already uses Docker.
- You need Docker Desktop features (GUI, Kubernetes cluster, Extensions).

Choose Podman when:

- Security is a priority (rootless, daemonless).
- You are on RHEL, Fedora, or CentOS Stream (pre-installed).
- You are developing for Kubernetes (pods, YAML generation).
- You need systemd integration for production services.
- Your organization's policy prohibits running daemons as

Conclusion

Key Takeaways

- Containers solve the “it works on my machine” problem by packaging an application with its dependencies into a **portable** unit.
- They achieve isolation and resource management via Linux kernel features: **namespaces** and **cgroups**.
- **Docker** popularized containers with a developer-friendly CLI, Dockerfile, and Docker Hub.
- **Docker Compose** enables declarative management of multi-service applications.
- **Podman** is a modern, daemonless, rootless alternative that is CLI-compatible with Docker and adds pod support, systemd integration, and Kubernetes YAML generation.
- The **OCI standard** ensures images and runtimes are interoperable across tools.

Further Resources and Useful Links

- **Docker Official Documentation:** The authoritative reference for Docker CLI, Dockerfile, and Compose.
  - https://docs.docker.com/
- **Podman Official Documentation:** Getting started guides, tutorials, and reference for Podman.
  - https://podman.io/docs
- **Docker Cheat Sheet (Collabnix):** A comprehensive cheat sheet with detailed examples.
  - https://dockerlabs.collabnix.com/docker/cheatsheet/

- **How to Optimize Docker Images (GeeksforGeeks):** Techniques like multi-stage builds to make images smaller and more secure.
  - https://www.geeksforgeeks.org/devops/how-to-optimize-docker-image/
- **Podman vs Docker (Red Hat):** An official comparison from the Podman developers.
  - https://www.redhat.com/en/topics/containers/what-is-podman
- **LinuxServer.io:** Community-maintained, high-quality container images for popular self-hosted applications.
  - https://www.linuxserver.io/